

Project Presentation of Research Paper

On

Comparative Analysis of Traditional and Docker-Based Deployment for Web Applications

Aryan Gupta
2210991380

Department of Computer Science and Engineering,
Chitkara University, Punjab

- Docker has become the default tool for packaging and deploying applications — it removes the 'works on my machine' problem.
- Linux-based performance consensus: Felter et al. (2015) from IBM Research confirmed <5% overhead; Potdar et al. (2020) confirmed near-native CPU; Mavridis & Karatza (2019) confirmed 20–30% memory savings.
- Cross-platform studies exist but are limited: Sladovic et al. (2024) found only 6% overhead on WSL2 — but used C compilation benchmarks, not web workloads.
- The WSL2 virtual Ethernet bridge adds per-packet overhead only visible under concurrent HTTP requests — compilation does not exercise this path.
- No prior study measured Docker-on-WSL2 under high-concurrency web workloads on modern hybrid-core processors (Intel Alder Lake).
- This paper tests Docker-on-WSL2 specifically under concurrent web workloads across 4 experiments on Windows 11 + i5-12500H.

- Sladovic et al. (2024) tested Docker on WSL2 using C compilation benchmarks — found only 6% overhead. But compilation is single-pass with no sustained network traffic.
- Sergeev et al. (2022) used arithmetic benchmarks — found marginal Windows/Linux differences. Same limitation: the virtual bridge was never continuously stressed.
- WSL2's virtual Ethernet bridge adds per-packet overhead only visible under concurrent HTTP request load — invisible to prior benchmark choices.
- Three specific gaps remain: (1) no WSL2 study used concurrent web workloads, (2) no study tested concurrency scaling 10–200 users.
- Three predictions derived from combined literature: memory efficiency WILL transfer (cgroup is kernel-internal); latency WILL be worse than 6% (bridge overhead); startup WILL exceed Khan's 2.69x (Hyper-V heavier than HyperKit).

- To detect whether the Linux-based Docker performance consensus holds on WSL2 under concurrent web application workloads.
- To design a multi-experiment framework: startup time (5 cold starts), concurrency scaling (10–200 users), memory scaling (3 matrix sizes), sustained execution (5 cycles).
- To compare findings directly against 6 prior papers: Felter (2015), Mavridis (2019), Potdar (2020), Sladovic (2024), Khan (2026), Haruna (2022).
- To develop real performance measurements across Native Windows and Docker/WSL2 environments on Intel i5-12500H Alder Lake.
- To generate a precise trade-off profile: what WSL2 costs (58% latency, 7.46x startup) vs what it gains (25% memory, better CPU use, 47.7% faster setup).
- To improve understanding over traditional compilation benchmarks — revealing overhead that single-pass benchmarks cannot detect.

- Hardware: Intel Core i5-12500H (Alder Lake — 4 P-cores + 8 E-cores = 16 threads), 16 GB DDR4, Windows 11 Pro, Docker Desktop v4.28 (WSL2 + Hyper-V), WSL2 cap: 7.51 GB.
- Native stack: Python 3.9 + numpy 1.24 + flask 2.3 (Windows native). Docker stack: python:3.9-slim image with same libraries.
- Workload: Flask web application serving concurrent matrix multiplication requests (float64) — stresses CPU and WSL2 virtual bridge simultaneously.
- Experiment 1: Startup time — 5 cold starts each (extends Khan 2026). Experiment 2: Concurrency scaling 10/50/100/200 users, 3 runs each (extends Potdar 2020).
- Experiment 3: Memory scaling at 500×500 / 1000×1000 / 2000×2000 matrices (extends Mavridis 2019). Experiment 4: Sustained execution 5 cycles under full load (tests Felter 2015 claim).
- Measurement: `time.perf_counter()` | docker stats API (cumulative CPU across all threads) | `typeperf` (Windows) | `psutil` RSS (memory).

Achievement



58.1%

Execution Latency
Overhead on WSL2

7.46x

Startup Penalty
vs Khan's 2.69x macOS

25.3%

Avg Memory Savings
(4 test conditions)

1605%

Docker CPU (all 16
threads) vs 92% Native

- Memory: 25.0%, 25.0%, 25.6%, 25.7% reduction across all 4 conditions — confirms Mavridis & Karatza's 20–30% range on WSL2.
- Latency: 58.9% overhead consistent across 10–200 concurrent users — ~10x higher than Sladovic's 6% (workload-dependent difference, not contradiction).
- Startup: 7.46x penalty on WSL2/Hyper-V vs Khan's 2.69x on macOS/HyperKit — WSL2 kernel initialization is heavier.
- CPU: Linux CFS on WSL2 saturates all 16 Alder Lake threads (1605%); Windows NT throttles background processes to ~1 core (92%).
- Deployment setup: Docker 47.7% faster (6.7 min vs 12.8 min) — layer caching confirmed on WSL2.
- Novel finding: Intel Thread Director (Windows) vs Linux CFS on hybrid P-core/E-core — not documented by any prior containerization study.

Key Features



- Multi-experiment approach: 4 experiments across startup, concurrency scaling, memory scaling, and sustained execution — not a single benchmark.
- Workload-appropriate: concurrent HTTP matrix requests continuously stress the WSL2 virtual Ethernet bridge — the exact path that compilation benchmarks skip.
- Literature-driven: every finding explicitly compared against 6 prior paper predictions — confirms, contradicts, or extends each one.
- Reconciles contradicting results: explains why Sladovic found 6% and we found 58% — workload dependency of WSL2 bridge overhead.
- Novel CPU scheduler finding: Linux CFS achieves full Alder Lake hybrid-core saturation on WSL2; Windows NT does not for background workloads.
- Real-world deployment guidance: concrete numbers (58% latency cost, 25% memory gain) for Windows developers making Docker-on-WSL2 decisions.

- Memory-constrained local development: 25.3% RAM savings on WSL2 — same as on native Linux, usable for local microservice hosting.
- Compute-intensive batch workloads: Linux CFS on WSL2 achieves near-complete Alder Lake saturation — better multi-core use than native Windows scheduling.
- Rapid deployment cycles: Docker layer caching gives 47.7% faster environment setup (6.7 min vs 12.8 min) on WSL2.
- Web services with known latency tolerance: 58% overhead is acceptable for background/batch services; not for latency-sensitive user-facing APIs.
- Cross-platform engineering research: establishing which prior Linux-based Docker findings transfer to WSL2 and which do not.
- Windows DevOps teams: first multi-experiment, literature-backed characterization of Docker-on-WSL2 for informed CI/CD pipeline decisions.

- Workload is compute-intensive only (matrix multiplication) — not representative of I/O-bound (PostgreSQL) or pure network-bound (file transfer) workloads.
- Small sample sizes: 3–5 cycles per experiment — no standard deviation or confidence intervals reported; descriptive statistics only.
- Single hardware platform: Intel i5-12500H Alder Lake on Windows 11 — results may differ on AMD, ARM, or older Intel symmetric-core architectures.
- No native Linux baseline on the same physical hardware — WSL2 attribution relies on prior literature comparison (Felter 2015, Sladovic 2024).
- Single-container only: Kubernetes orchestration, Kubelet overhead, and sidecar proxy costs not measured — may affect memory efficiency at scale.
- Specific to Docker Desktop v4.28 + WSL2 + Hyper-V — other WSL2 versions, Windows builds, or Docker configurations may yield different overhead.

Conclusion



- Docker-on-WSL2 does NOT perform as prior cross-platform studies suggested — the answer depends heavily on workload type.
- Sladovic (2024) found 6%, Sergeev (2022) found marginal differences — both valid for their benchmarks. Our concurrent web workload found 58.1% — ~10x higher. This is workload dependency, not contradiction.
- Memory efficiency IS workload-independent and transfers fully: 25.3% average reduction confirms Mavridis & Karatza's cgroup mechanism works in WSL2.
- Startup on WSL2 (7.46x) is significantly worse than macOS Docker Desktop (2.69x) — Hyper-V initialization is heavier than HyperKit.
- CPU finding was not predicted by any prior work: Linux CFS achieves near-complete saturation of all 16 Alder Lake threads; Windows NT cannot for background workloads.
- Practical summary: Docker-on-WSL2 costs ~58% latency under web load, gains 25% memory efficiency, 47.7% faster setup — a specific, knowable trade-off.

Thank You